Primeira Aplicação no K8S

Do Dockerfile ao Deployment, com Rollout e Rollback

Ada · DE-OP-011 Orquestração · Kubernetes | Aula 04

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada = aula top!

INTERVALO • 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

TRABALHO FINAL

Entrega Final!

O desafio

- Suba no **minikube** um Pod com uma API e um Banco de Dados à sua escolha.
- A API deve fazer um **CRUD para produtos**.

TABELA produtos

Id	—	INT
Nome	—	VARCHAR(50)

O que significa CRUD

- **C** — **Create** — Criar
- **R** — **Read** — Ler
- **U** — **Update** — Atualizar
- **D** — **Delete** — Excluir

Atualize a versão do Pod de forma **declarativa**, demonstrando:

Rolling Update | **Recreate**

Objetivos de hoje

- Entender o papel do Docker Hub no fluxo
- Construir uma imagem Docker e publicá-la
- Fazer deploy da aplicação via Deployment
- Verificar Pods e logs
- Dominar Rollout e Rollback

Pré-requisitos

- Docker instalado
- kubectl e Minikube (das aulas anteriores)
- Conta no Docker Hub
- Cluster Minikube rodando (minikube start)

1. Docker Hub e a imagem

Onde as imagens ficam guardadas



Por que precisamos de um registry

- O K8S precisa BAIXAR a imagem para rodar o container
- A imagem precisa estar em um repositório acessível
- Docker Hub: registro público gratuito de imagens
- Alternativas: ECR (AWS), GCR, Azure ACR, registries privados

A aplicação e o Dockerfile

app.rb (loga Hello World) + Dockerfile

```
# app.rb
loop do
  puts 'Hello world!'
  STDOUT.flush
  sleep 1
end

# Dockerfile
FROM ruby:3.4.10-alpine   # imagem base com Ruby
COPY . /app               # copia o código para /app
WORKDIR /app              # define o diretório de trabalho
CMD ["ruby", "app.rb"]    # comando executado ao subir
```

Build e push da imagem

MÃO NA MASSA

Autentique no Docker Hub:

```
$ docker login
```

Construa a imagem (troque <username> pelo seu usuário):

```
$ docker build . -f app.dockerfile -t <username>/primeira-aplicacao:1.0
```

Publique a imagem no Docker Hub:

```
$ docker push <username>/primeira-aplicacao:1.0
```

Build e push da imagem

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-04> docker login
Authenticating with existing credentials... [Username: evertondevbr]

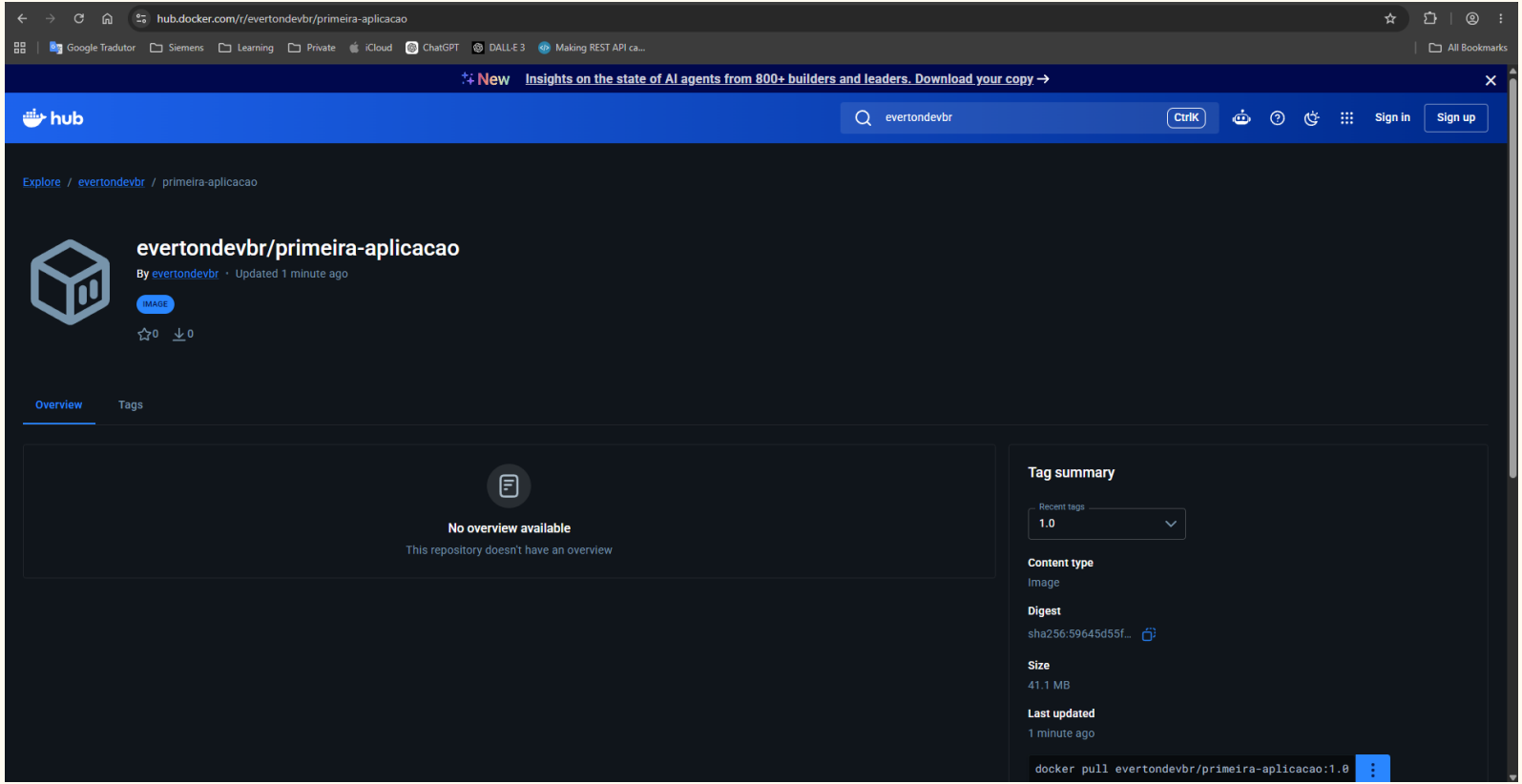
i Info → To login with a different account, run 'docker logout' followed by 'docker login'
```

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-04> docker build . -f app.dockerfile -t evertondevbr/primeira-aplicacao:1.0
[+] Building 30.8s (8/8) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from app.dockerfile                                                                          0.0s
 => => transferring dockerfile: 115B                                                                                              0.0s
 => [internal] load metadata for docker.io/library/ruby:3.4.10-alpine                                                             0.9s
 => [internal] load .dockerignore                                                                                                 0.0s
 => => transferring context: 2B                                                                                                   0.0s
 => [internal] load build context                                                                                                 0.1s
 => => transferring context: 222B                                                                                                 0.0s
 => [1/3] FROM docker.io/library/ruby:3.4.10-alpine@sha256:c5a5064d190055633011c03aa800170cc36945ff3afb5f6c915329f92d6f1e00     29.2s
 => => resolve docker.io/library/ruby:3.4.10-alpine@sha256:c5a5064d190055633011c03aa800170cc36945ff3afb5f6c915329f92d6f1e00      0.0s
 => => sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 3.85MB / 3.85MB                                    3.0s
 => => sha256:39b52365d7706a257f6d7dfad21a886daf80bf1a43fd8eb064cc01faa55a0c1c 139B / 139B                                        0.3s
 => => sha256:16b23947d7a97805dc145cb87cb5552fe8093e40af219e58a810b8cb4cb4b681 190B / 190B                                        0.4s
 => => sha256:c4fc3d3e01c15b412bc0c612fc046c8b71bb6b9e97c012832b0e070898bd5543 39.25MB / 39.25MB                                  28.2s
 => => extracting sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4                                         0.1s
 => => extracting sha256:16b23947d7a97805dc145cb87cb5552fe8093e40af219e58a810b8cb4cb4b681                                         0.0s
 => => extracting sha256:c4fc3d3e01c15b412bc0c612fc046c8b71bb6b9e97c012832b0e070898bd5543                                         0.8s
 => => extracting sha256:39b52365d7706a257f6d7dfad21a886daf80bf1a43fd8eb064cc01faa55a0c1c                                         0.0s
 => [2/3] COPY . /app                                                                                                             0.1s
 => [3/3] WORKDIR /app                                                                                                            0.0s
 => exporting to image                                                                                                            0.3s
 => => exporting layers                                                                                                           0.2s
 => => exporting manifest sha256:b7d72f58458289dda0e0ebf3a9c8e5fb604e893b99d345317e3e3a1a2c149053                                 0.0s
 => => exporting config sha256:04aebcf55c57b48299b37ee2f4fbe8a29f1c2af9aa637fcbf30db53259293ae6                                   0.0s
 => => exporting attestation manifest sha256:bbcbe7dfc8e1835c72929a6f3de1f4db5a65438cb1446f7535569d87dc901e63                     0.0s
 => => exporting manifest list sha256:59645d55f15ce547e1f55c66d2daa7518fcbb590bd770a09681d30d0570683d6                            0.0s
 => => naming to docker.io/evertondevbr/primeira-aplicacao:1.0                                                                    0.0s
 => => unpacking to docker.io/evertondevbr/primeira-aplicacao:1.0                                                                 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/id91m0ibs48iif7y6rdk2c979
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-04> docker push evertondevbr/primeira-aplicacao:1.0
The push refers to repository [docker.io/evertondevbr/primeira-aplicacao]
39b52365d770: Pushed
24fb5af712bd: Pushed
4f4fb700ef54: Pushed
55afa1ecc21d: Pushed
c4fc3d3e01c1: Pushed
16b23947d7a9: Pushed
82247f981712: Pushed
1.0: digest: sha256:59645d55f15ce547e1f55c66d2daa7518fcbb590bd770a09681d30d0570683d6 size: 856
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-04>
```

Fonte da imagem: Ada Tech

Build e push da imagem (cont.)



Fonte da imagem: Ada Tech

2. Deploy no cluster

Colocando a aplicação no ar

Manifesto do Deployment

primeira-aplicacao.yaml — 3 réplicas no namespace 'aplicacao'

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: primeira-aplicacao
  namespace: aplicacao      # namespace dedicado
  labels:
    app: ruby
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ruby
  template:
    metadata:
      labels:
        app: ruby
    spec:
      containers:
      - name: primeira-aplicacao
        image: <username>/primeira-aplicacao:1.0  # sua imagem
        ports:
        - containerPort: 80
```

Aplicando e verificando

MÃO NA MASSA

Crie o namespace da aplicação:

```
$ kubectl create ns aplicacao
```

Aplique o Deployment:

```
$ kubectl apply -f primeira-aplicacao.yaml
```

Veja os 3 Pods do namespace:

```
$ kubectl get pods -n aplicacao
```

Confira os logs de um Pod (Hello world a cada 1s):

```
$ kubectl logs -n aplicacao <nome-do-pod>
```

Aplicando e verificando

```
PS C:\UserData\        Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl create ns aplicacao
namespace/aplicacao created
PS C:\UserData\        Documents\Private\mine\ada\orquestracao-k8s\aula-04>
PS C:\UserData\        Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl apply -f primeira-aplicacao
error: the path "primeira-aplicacao" does not exist
PS C:\UserData\        Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl apply -f primeira-aplicacao.yaml
deployment.apps/primeira-aplicacao created
PS C:\UserData         :\Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-bxwcw   1/1     Running   0          48s
primeira-aplicacao-564b64f5b4-mm54f   1/1     Running   0          14s
primeira-aplicacao-564b64f5b4-qkb5k   1/1     Running   0          13s
PS C:\UserData\        .Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl logs -n aplicacao primeira-aplicacao-564b64f5b4-qkb5k
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
Hello world!
```

Fonte da imagem: Ada Tech

3. Rollout e Rollback

Atualizar e reverter com segurança

Rollout: histórico de alterações

- Toda mudança em um Deployment é um Rollout
- É possível acompanhar o status em tempo real
- É possível consultar o histórico de revisões
- kubectl rollout status / kubectl rollout history

Provocando um erro e revertendo

MÃO NA MASSA

Acompanhe o status do rollout:

```
$ kubectl rollout status -w deployment/primeira-aplicacao -n aplicacao
```

Veja o histórico de revisões:

```
$ kubectl rollout history deployment/primeira-aplicacao -n aplicacao
```

Force um erro: aponte para uma imagem inexistente

```
$ kubectl set image -n aplicacao deployment/primeira-aplicacao primeira-aplicacao=usuario/imagem-inexistente:1.0
```

Observe o status ImagePullBackOff:

```
$ kubectl get pods -n aplicacao
```

Reverta para a versão anterior estável:

```
$ kubectl rollout undo deployment/primeira-aplicacao -n aplicacao
```

Rollout: histórico de alterações

```
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl rollout status -w deployment/primeira-aplicacao -n aplicacao
deployment "primeira-aplicacao" successfully rolled out
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl rollout history deployment/primeira-aplicacao -n aplicacao
deployment.apps/primeira-aplicacao
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl set image -n aplicacao deployment/primeira-aplicacao primeira-aplicacao=evertondevbr/imagem-inexistente:1.0
deployment.apps/primeira-aplicacao image updated
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl get pods -n aplicacao
NAME                                 READY   STATUS         RESTARTS   AGE
primeira-aplicacao-564b64f5b4-bxwcw   1/1     Running        0          11m
primeira-aplicacao-564b64f5b4-mm54f   1/1     Running        0          10m
primeira-aplicacao-564b64f5b4-qkb5k   1/1     Running        0          10m
primeira-aplicacao-857f4d5cb6-tbqv9   0/1     ErrImagePull   0          36s
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl get pods -n aplicacao
NAME                                 READY   STATUS             RESTARTS   AGE
primeira-aplicacao-564b64f5b4-bxwcw   1/1     Running            0          11m
primeira-aplicacao-564b64f5b4-mm54f   1/1     Running            0          10m
primeira-aplicacao-564b64f5b4-qkb5k   1/1     Running            0          10m
primeira-aplicacao-857f4d5cb6-tbqv9   0/1     ImagePullBackOff   0          53s
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl get pods -n aplicacao
NAME                                 READY   STATUS             RESTARTS   AGE
primeira-aplicacao-564b64f5b4-bxwcw   1/1     Running            0          12m
primeira-aplicacao-564b64f5b4-mm54f   1/1     Running            0          11m
primeira-aplicacao-564b64f5b4-qkb5k   1/1     Running            0          11m
primeira-aplicacao-857f4d5cb6-tbqv9   0/1     ImagePullBackOff   0          109s
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl rollout undo deployment/primeira-aplicacao -n aplicacao
deployment.apps/primeira-aplicacao rolled back
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl get pods -n aplicacao
NAME                                 READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-bxwcw   1/1     Running   0          13m
primeira-aplicacao-564b64f5b4-mm54f   1/1     Running   0          12m
primeira-aplicacao-564b64f5b4-qkb5k   1/1     Running   0          12m
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl rollout history deployment/primeira-aplicacao -n aplicacao
deployment.apps/primeira-aplicacao
REVISION  CHANGE-CAUSE
1         <none>
3         <none>
4         <none>

PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-04> kubectl rollout status -w deployment/primeira-aplicacao -n aplicacao
deployment "primeira-aplicacao" successfully rolled out
```

Fonte da imagem: Ada Tech

Recapitulando a Aula 04

- ✓ Fluxo completo: código → imagem → Docker Hub → Deployment
- ✓ Deployment mantém as réplicas no ar (self-healing)
- ✓ Logs ajudam a verificar a aplicação
- ✓ Rollout registra o histórico; Rollback reverte com segurança
- ✓ Próxima aula: estratégias de deploy e limitação de recursos

